EIE1005 Fundamental AI and Data Analytics

AI and Data Analytics in the Virtual World



Richard Pang

Tel: 27664745
Email: richard.pang@polyu.edu.hk

Contents

- Development of game AI
- Reinforcement learning for game AI
- Developing game AI with Unity ML-Agents
- AI and data analytics in the Metaverse

References

- G.N. Yannakakis and J. Togelius, *Artificial intelligence and games*, Springer, 2018
- T. Simonini and O. Saneviero, *The Hugging Face Deep Reinforcement Learning Class,* https://github.com/huggingface/deep-rl-class
- A. Majumder, *Deep Reinforcement Learning in Unity: With Unity ML Toolkit*. Berkeley, CA: Apress, 2020.
- D. Buckley, “Unity ML-Agents Tutorials – Complete Guide,” *GameDev Academy*, 2022 (online resource).
- T. Simonini, “An Introduction to Unity ML-Agents,” Towards Data Science, 2020 (online resource)

Part II

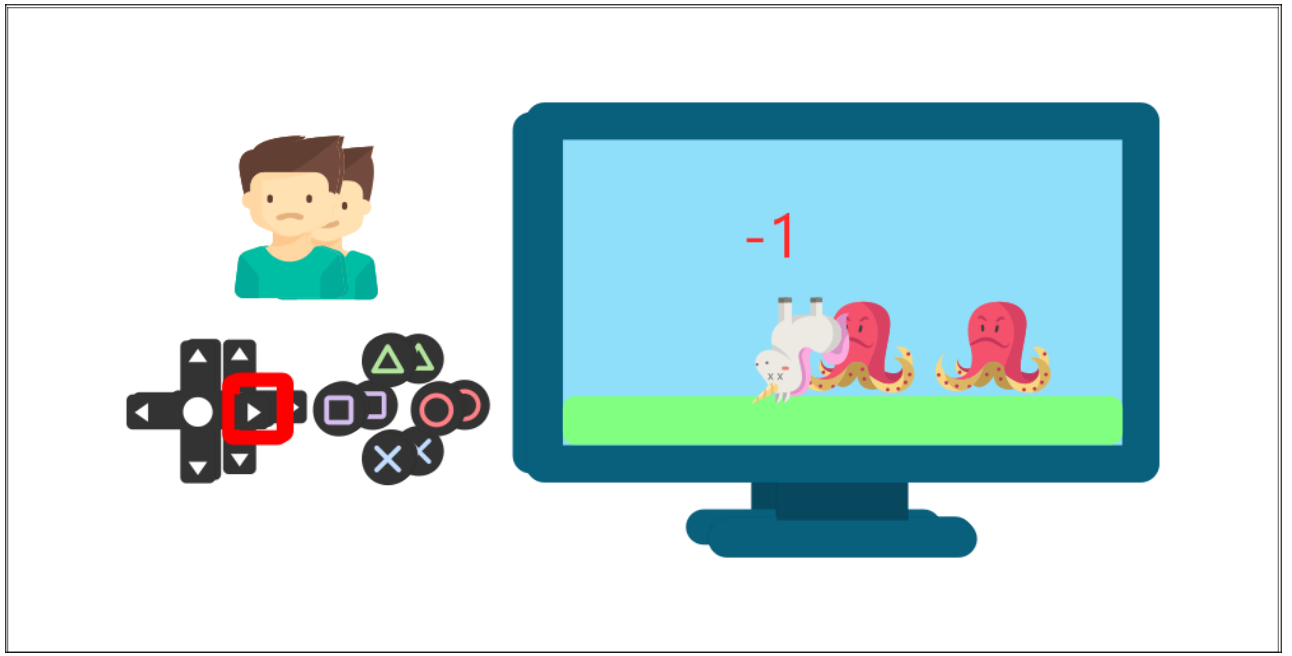
Reinforcement Learning for Game AI

What is Reinforcement Learning?

- Reinforcement Learning – an agent (an AI) will learn from the environment by interacting with it (through trial and error) and receiving rewards (negative or positive) as feedback for performing actions
- Commonly used in game AI development
- Different from supervised and unsupervised learning
  - Supervised learning: A child is put in a school to learn from teachers who always give answers to students
  - Unsupervised learning: A child is put in a school and learns only from books and exercises with no teachers and answers
  - Reinforcement learning: A child is put in the working place to learn in the working environment!

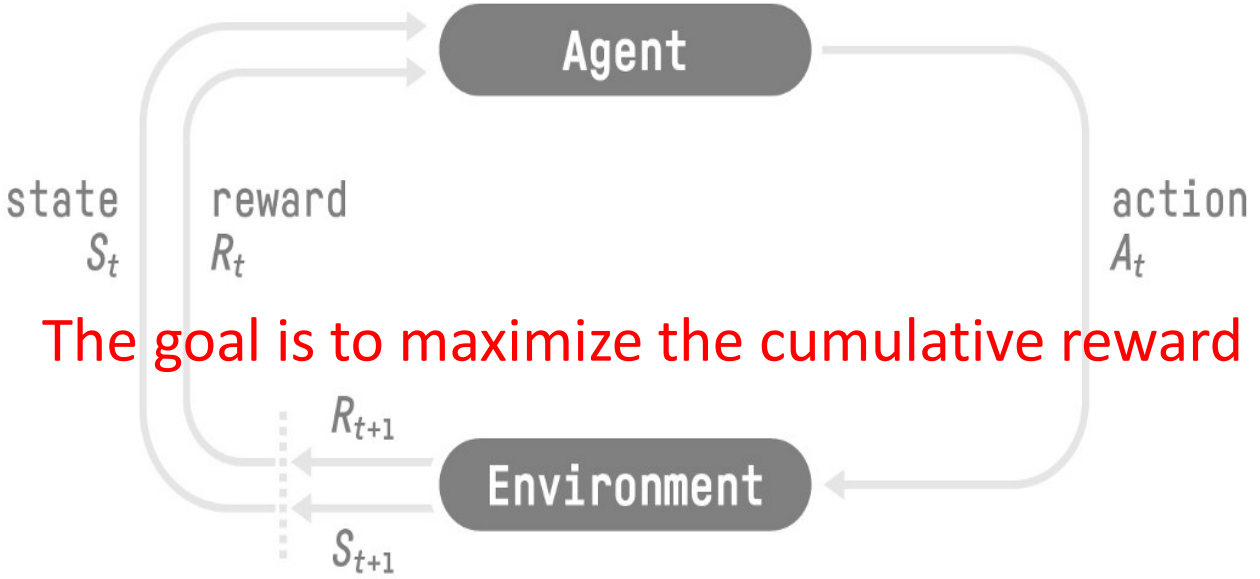
RL Comes from Natural Experiences

- By interacting with the environment through trial and error, a player can get better and better at playing a game without any supervision
- It is how humans learn though interaction



The Reinforcement Learning Framework

- Agent
  - An NPC, a moving object, etc.
- Environment
  - The game world
- State (S_t)
  - The state of the Agent at time unit t
- Action (A_t)
  - The action made by the Agent at time unit t
- Reward (R_t)
  - The reward the Environment gives to the Agent at time unit t

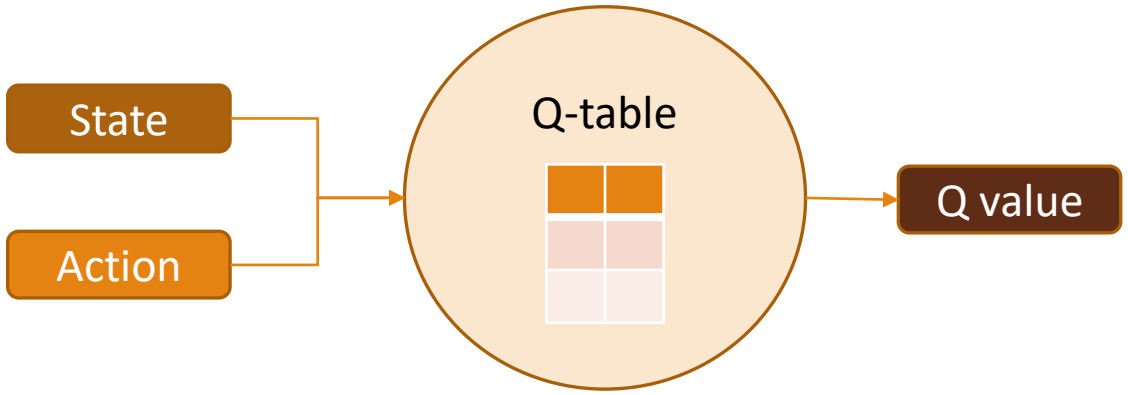


The goal is to maximize the cumulative reward

This RL loop outputs a sequence of state S_t, action A_t, reward R_t, next reward R_{t+1}, and new state S_{t+1}

Introduction to Q-Learning

- Q-Learning is a reinforcement learning method that will find the next best action for an agent, given a current state
- Q comes from “the Quality” of that action at that state
- A Q-table is kept in the method that indicates the quality (Q value) of every move of the agent at every state
- The Q-table is updated for each move. With sufficient exploration and training, the Q-table converges towards the optimal action values.



Given a state and action as input, the Q-table is trained by the Q-Learning algorithm to estimate the Q value of that state and action with respect to the final goal

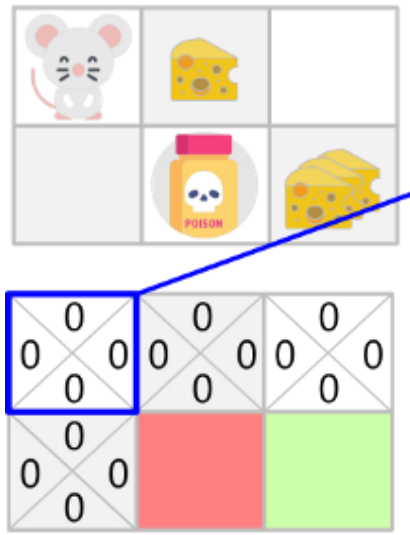
Q-Learning Example: Maze



- Goal: To train the mouse to automatically find the path to eat maximum amount of cheese and avoid poisoned
- There are 6 states (each box represents one state)
- And 4 actions (up, down, right, left by one box) for each state
- A Q-table is trained to indicate the value of each state and action

Step 1 - Initialize the Q-table

- The Q-table is first initialized to all zeros



Actions

States

	←	→	↑	↓
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0

Step 2 – Choose an Action

- How should the mouse move? Two strategies: Exploration or exploitation

Exploration

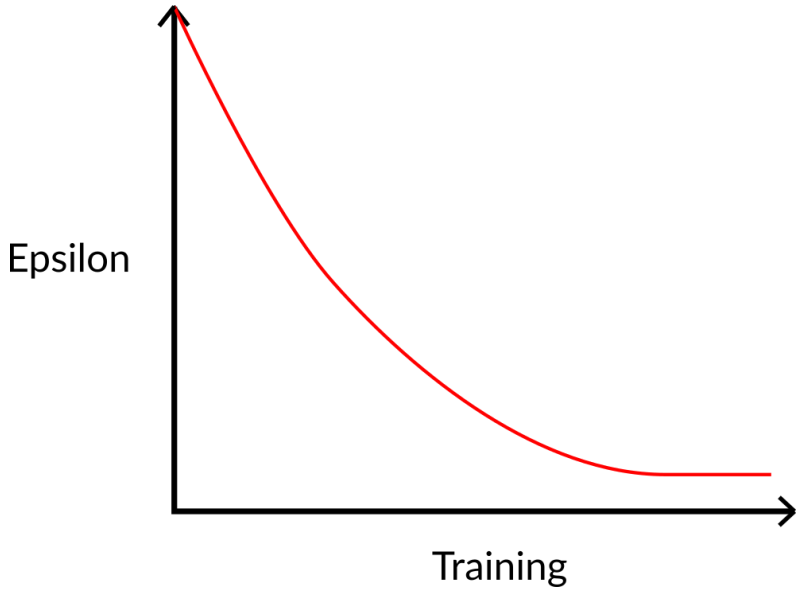
- Trying random actions in order to find more information about the environment
- More suitable at initial stage when the Q-table is not sufficiently smart

Exploitation

- Using known information to maximize the reward
- More suitable at the later stage when the Q-table is sufficiently smart

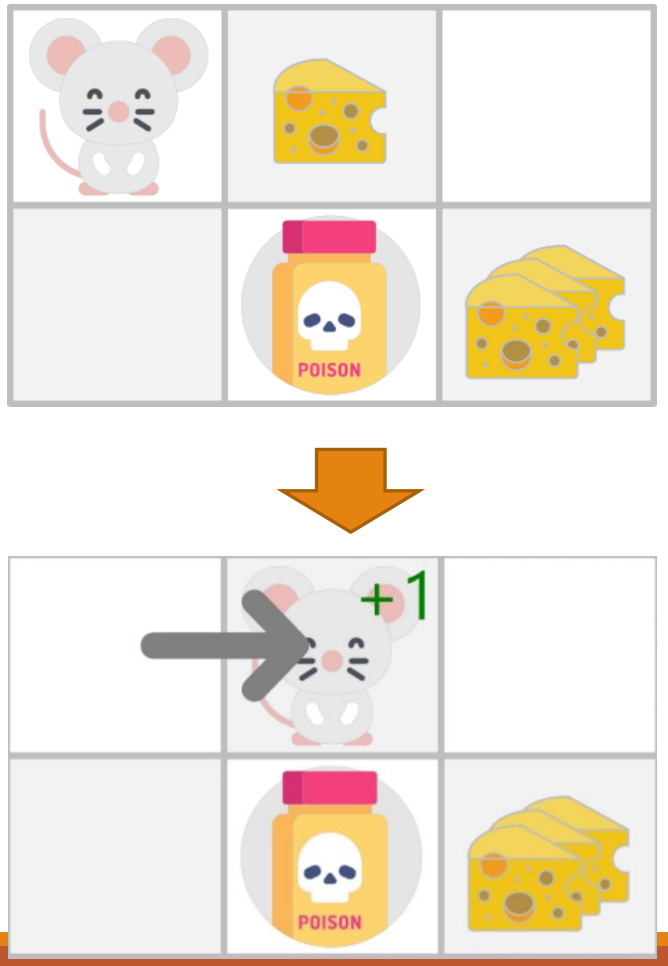
Step 2 – Choose an Action

- Control by a parameter epsilon $0 \leq \varepsilon \leq 1$
  - Refer to the probability ε that exploration (trying random action) will be performed
- At the beginning of the training, ε is large
  - Most of the time the mouse will explore randomly (exploration)
- After some time of training, ε becomes small
  - The mouse will more follow the values in the Q-table to determine the action (exploitation)



Step 3 – Perform the Action and Get Reward

- The reward system of the game is as follows:
  - +0: Going to a state with no cheese in it
  - +1: Going to a state with a small cheese in it
  - +10: Going to the state with big pile of cheese
  - -10: Going to the state with the poison and thus die
- Assume at the beginning, the mouse is at the left upper corner
- Since ε is large at the beginning, the mouse is in exploration mode. It randomly moves one step to the right and eat the cheese (thus gets 1 point)



Step 4 – Update Q-table

- Based on the action and reward, the value of the Q-table of state S_t is updated following the Bellman equation

	←	→	↑	↓
	0	0.1	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0

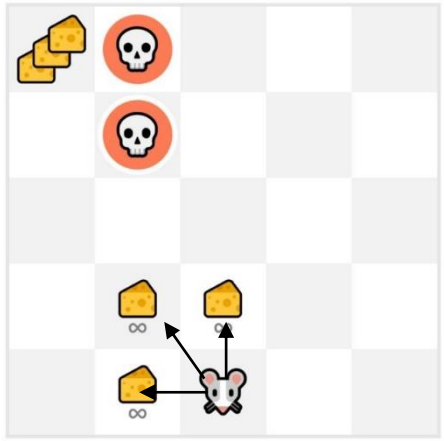
- The Q-value $Q(S_{0,}, A_0)$ is changed from 0 to 0.1
- The higher than Q-value, the more correct the mouse makes that move at that state
- The value 0.1 is determined by the Bellman equation

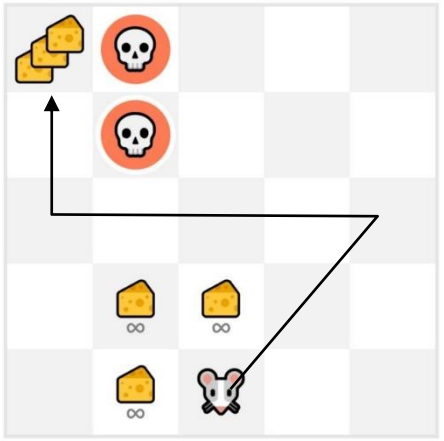
The Bellman equation

- The Bellman equation

New Q-value = Original Q-value +

{ Reward of the action + Possible increase of Q-value due to this action}





- Does not only rely on the immediate reward, look also to the future
- Ask the question: Would the current action bring to higher Q-value in the future based on experience?
- If yes, New Q-value has a higher value. Thus endorse the current action
- If no, New Q-value has a lower value. Thus discredit the current action

The Bellman equation

- A simple implementation of the Bellman equation

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \eta \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

The maximum possible gain in Q-value due to the current action

Updated Q-value of state S_t action A_t

Original Q-value of state S_t action A_t

Reward we get due to the current move, i.e., 1 point

The maximum possible Q-value after going to the next state from the current state

Original Q-value of state S_t action A_t

Learning rate η :

- Determine how fast we want the Q-table to learn
- Performance may not be good if learning too fast
- We set its value to 0.1 in this example

Discount parameter γ :

- The Q-value of the next state should have less influence on the current Q-value
- Multiply with γ that has a value less than 1 to reflect this
- We set it to 0.99 in this example

Step 4 – Update Q-table

$$Q(S_0, A_0) \leftarrow Q(S_0, A_0) + 0.1\left[R_1 + 0.99 \max_a Q(S_1, a) - Q(S_0, A_0)\right]$$

	←	→	↑	↓
S_0	0	0.1	0	0
S_1	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0

$$Q(S_0, Right) = 0 + 0.1 \times [1 + 0.99 \times 0 - 0] = 0.1$$

$$\max_a Q(S_1, a) = 0$$

- If $\gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) > 0$ means it is possible to have a gain in Q-value when taking the current action
- So, reward the move by giving a larger Q-value to the current (state, action)
- Vice versa

One More Action – Step 1

- The reward system of the game is as follows:
  - +0: Going to a state with no cheese in it
  - +1: Going to a state with a small cheese in it
  - +10: Going to the state with big pile of cheese
  - -10: Going to the state with the poison and thus die
- Since ε is large at the beginning, the mouse is in exploration mode. It randomly moves one step down and drink the poison (thus gets -10 points)



Step 2 – Update Q-table

$$Q(S_1, A_1) \leftarrow Q(S_1, A_1) + 0.1\left[R_1 + 0.99 \max_a Q(S_2, a) - Q(S_1, A_1)\right]$$

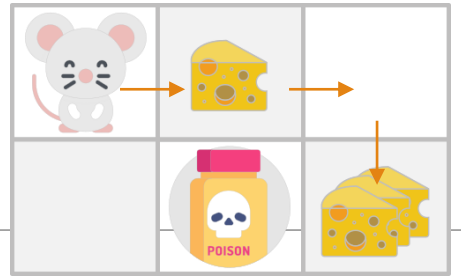
	←	→	↑	↓
S_0	0	0.1	0	0
S_1	0	0	0	-1
	0	0	0	0
	0	0	0	0
S_2	0	0	0	0
	0	0	0	0

$$Q(S_1, down) = 0 + 0.1 \times [-10 + 0.99 \times 0 - 0] = -1$$

- The Q-value -1 here indicates that it is not a good idea to take that action
- Next time when moving to that position, the mouse knows it should avoid making the same mistake

$$\max_a Q(S_2, a) = 0$$

At the End …



The value becomes so small (-10) to indicate the mouse should not move downwards at that state

	←	→	↑	↓
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0

Training

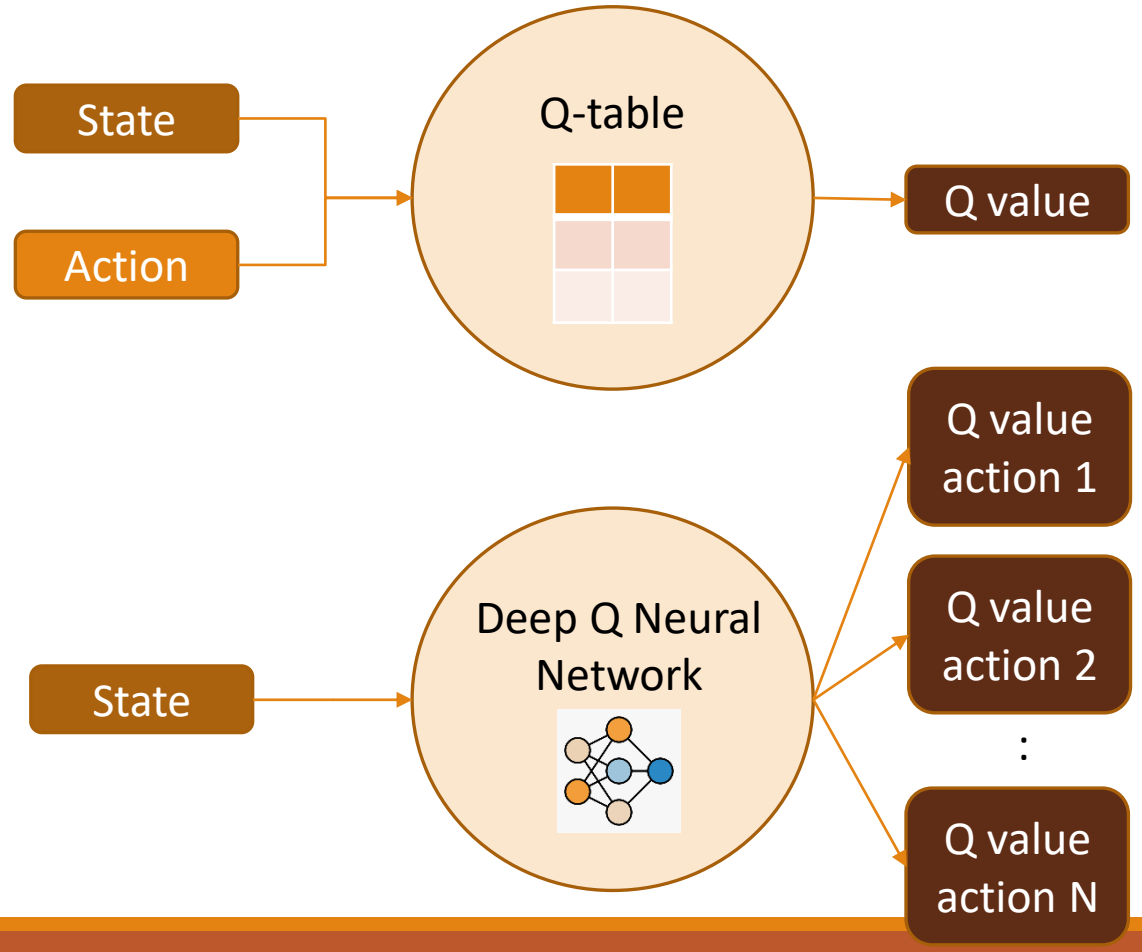
	←	→	↑	↓
	0	10.8	0	0
	0	9.9	0	-10
	0	0	0	10
	-10	0	0	0
	0	0	0	0
	0	0	0	0

Right moves

- After training for a while, the Q-table becomes smart enough to inform the right action
- The mouse will incline to use exploitation (by referring to the Q-table) rather than exploration (randomly) when choosing the move

Deep Q-Learning

- Q-Learning works well with small number of states and actions
- The Q-table will be incredibly large for modern computer games
- Rather using table lookup, deep Q-learning uses a deep neural network to generate the Q-value
- Approximate, given a state, the different Q-values for each possible action at that state
- Deep Q-Network can handle large state spaces by approximating Q-values with a neural network; GPUs accelerate both training and inference



Workshop 3 – OpenAI Gym's Frozenlake

- OpenAI Gym
  - An open-source Python library for developing and comparing reinforcement learning algorithms
    - Provide a standard application program interface (API) to communicate between learning algorithms and environments
    - Provide a standard set of environments compliant with that API (Frozenlake is one of them)
- Goal
  - To train an agent to walk on a frozen lake to the Goal and avoid falling into any Holes
  - Ice is slippery; the agent only has 1/3 chance to follow your instructed direction to move

